

FILE I/O

BY SUJAL SHRESTHA

FILE HANDLING

File Handling in Python allows you to read from and write to files. Python provides several built-in functions and methods to interact with files, including opening, reading, writing, and closing files.

Common File Operations in Python:

1. Opening a File
2. Reading from a File
3. Writing to a File
4. Closing a File
5. File Modes
6. File Operations Using Context Managers (with keyword)

OPENING A FILE

To perform any file operation, you first need to open the file using the built-in `open()` function.

Syntax:

```
python
```

```
file = open('filename', 'mode')
```

Note:

- 'filename' is the name of the file (and its path if it's not in the same directory).
- 'mode' specifies the mode in which the file is opened (e.g., read, write).

FILE MODES

The file modes define how you can open and interact with a file:

- 'r': Read mode (default). Opens the file for reading. File must exist
- 'w': Write mode. Opens the file for writing (creates the file if it doesn't exist or overwrites the file if it exists).
- 'a': Append mode. Opens the file for appending (creates the file if it doesn't exist).
- 'b': Binary mode (e.g., 'rb', 'wb').
- 'x': Exclusive creation. Creates a new file, but fails if the file already exists.
- 't': Text mode (default). Use with 'rt', 'wt'.
- '+': read + write combined.

FILE MODES

+ mode

- 'r+': Read and write. File must exist
- 'w+': Write and read (creates the file if it doesn't exist or overwrites the file if it exists).
- 'a+': Append and read (creates the file if it doesn't exist)
- 'x+': Create and read/ write, but fails if the file already exists.

READING FROM A FILE

You can read files using the following methods:

- `read()`: Reads the entire file as a string.
- `readline()`: Reads one line at a time.
- `readlines()`: Reads all lines and returns them as a list.

READING FROM A FILE

```
# Open file in read mode  
file = open("example.txt", "r")
```

```
# Read the entire file  
content = file.read()  
print(content)
```

```
# Close the file  
file.close()
```

WRITING TO A FILE

To write data to a file, you can use the `write()` or `writelines()` methods.

- `write()` writes a string to the file.
- `writelines()` writes a list of strings to the file.

WRITING TO A FILE

```
# Open file in write mode
file = open("example.txt", "w")

# Write content to the file
file.write("Hello, World!\n")
file.write("This is Python file handling.")

# Close the file
file.close()
```

CLOSING A FILE

It is important to close the file after you're done working with it to free up system resources.

Syntax:

```
file.close()
```

FILE OPERATIONS USING CONTEXT MANAGERS (WITH KEYWORD)

Using the with statement automatically handles file closing, making the code more concise and cleaner. When the block inside with finishes execution, the file is automatically closed, even if an exception occurs.

Example:

```
with open("example.txt", "r") as file:
```

```
    content = file.read()
```

```
    print(content)
```

QUESTIONS

- Suppose, intro.txt file has “Hello, I am Entity.”. Write a Python program to open a file named intro.txt in read mode and display its contents.
- Create a program that writes the sentence "Python is fun!" into a file called notes.txt.
- Write a program to append the line "Learning File I/O" into the existing file notes.txt.
- Write a program to read a file notes.txt line by line and print each line separately.
- Write a program to count the total number of characters in a file notes.txt

QUESTIONS

THANK YOU



FOR COMING